PROTOCOLO DE PREVIO FINAL

GhostKitchen Hunter: cazadores de cocinas fantasma en Cúcuta

Proyecto	GhostKitchen Hunter: cazadores de cocinas fantasma en Cúcuta
Estudiante	Nicolás Andrés Sánchez Villamizar
Asignatura	Patrones de Diseño de Software
Docente	Jonathan Rey Castillo
Institución	Universidad de Santander (UDES), Cúcuta
Repositorio	https://github.com/NicolasSnchz/sanchez-ghostkitchen
Patrón asignado	Command: encapsular solicitudes como objetos con historial

Contenido

Sección 1. Descripción del Sistema e Identificación de Principios
Sección 2. Casos de Uso de Mayor Impacto
Sección 3. Modelo de Clases
Sección 4. Propuesta de Aplicación de Patrones de Diseño
Sección 5. Arquitectura del Sistema
Sección 6. Escenarios de Prueba
Sección 7. Métricas de Calidad y Plan de Refactorización

El código, el README con el diseño propuesto, las decisiones de diseño, los ADR, los diagramas fuente y las evidencias de GitHub Actions están en el repositorio indicado arriba.

Sección 1 — Descripción del Sistema e Identificación de Principios

Esta sección establece la base del proyecto. El equipo debe demostrar comprensión del problema antes de diseñar, y analizar qué principios fundamentales guiarán las decisiones de diseño.

1.1 Ficha del Sistema

Campo	Descripción
Nombre del sistema	GhostKitchen Hunter
Problema central que resuelve	En Cúcuta funcionan muchas cocinas fantasma: negocios de comida sin local ni letrero que cocinan en casas o garajes y solo venden por domicilio (Rappi, DiDi Food, WhatsApp o reparto propio). Son difíciles de encontrar: no aparecen en los mapas, sus horarios son cambiantes y su reputación depende del voz a voz. El comensal no tiene dónde descubrirlas ni opinar sobre ellas, y la cocina no gana visibilidad. GhostKitchen Hunter las reúne en un catálogo con mapa y convierte el encontrarlas en un juego: cada cocina descubierta es una caza que da puntos según su rareza, con bonos, retos, insignias, niveles y un ranking público.
Stakeholders primarios	Cazadores: comensales jóvenes que exploran, cazan, reseñan y compiten en el ranking. Administrador del catálogo: registra, edita, activa o desactiva cocinas y crea retos.
Stakeholders secundarios	Dueños de las cocinas fantasma: reciben visibilidad y opiniones (no tienen cuenta en esta versión). Plataformas de domicilio: aparecen como canales de pedido de cada cocina. Docente evaluador: revisa diseño, patrones, pruebas y métricas.
Alcance (dentro)	Registro e ingreso con roles CAZADOR y ADMIN. Explorador con mapa, búsqueda sin tildes, filtros y cuatro órdenes (cercanía, calificación, popularidad, novedad). Cazar una cocina con cálculo de puntos (base por rareza más bonos de primer cazador, racha y hora fantasma). Reseñas de 1 a 5 estrellas, deshacer la última acción durante 15 minutos. Retos con meta y recompensa, nueve insignias y cinco niveles automáticos. Panel del cazador, ranking, administración del catálogo y de retos, API REST de lectura y datos de demostración.
Alcance (fuera — exclusiones)	Pedidos y pagos: se hacen por el canal de cada cocina. Comprobar que el cazador de verdad compró (se confía en su palabra, con límite de 5 cazas al día). Cuentas para dueños de cocinas, aplicación móvil nativa y notificaciones por correo o push. Rehacer acciones deshechas y base de datos persistente de producción (se usa H2 en memoria).

1.2 Análisis de Principios SOLID

Para cada principio, indicar cómo se aplica en el sistema propuesto o justificar por qué no aplica. Si no se aplica, señalar qué riesgo implicaría ignorarlo.

Principio	¿Aplica?	Cómo se aplica o justificación de no aplicabilidad
S — Single Responsibility	Sí	Cada clase tiene un solo motivo de cambio: RegistroCaza solo aplica las reglas de la caza, HistorialAcciones solo ejecuta, registra y deshace comandos, cada bono de puntos es una clase con una sola regla y los controladores solo traducen HTTP. La regla GodClass de PMD falla el build si una clase concentra demasiado.
O — Open/Closed	Sí	Se extiende sin modificar: un bono nuevo es otra subclase de

Principio	¿Aplica?	Cómo se aplica o justificación de no aplicabilidad
		BonificacionPuntos; un orden nuevo es otro bean CriterioOrden que SelectorCriterioOrden registra solo; una reacción nueva al progreso es otro ObservadorProgreso. Ninguno obliga a tocar las clases existentes.
L — Liskov Substitution	Sí	Cualquier decorador sustituye a CalculadoraPuntos porque calcular() es final en la clase base y las subclases solo deciden aplica() y bonificar(). Cazador y Administrador sustituyen a Usuario en la autenticación y cualquier CriterioOrden funciona en el selector. PU-002 lo verifica.
I — Interface Segregation	Sí	Interfaces pequeñas y cohesivas: Comando (4 métodos), CalculadoraPuntos (1), ObservadorProgreso (1), CriterioOrden (2 más un método por defecto). Un observador no conoce los comandos ni el cálculo de puntos.
D — Dependency Inversion	Sí	Los módulos de alto nivel dependen de abstracciones: HistorialAcciones usa Comando y FabricaComandos; RegistroCaza recibe CalculadoraPuntos; el reloj se inyecta como Clock. La composición concreta vive solo en ConfiguracionDominio y ArchUnit impide que una capa baja dependa de una alta.

1.3 Compromisos de Calidad desde el Diseño

Identificar al menos 2 code smells que el equipo se compromete a evitar durante el desarrollo

Code smell comprometido a evitar	Estrategia de prevención
God Class (un servicio que lo hace todo)	Una clase por responsabilidad y una fachada (PanelCazadorFacade) que solo coordina. PMD con la regla GodClass y el límite de complejidad ciclomática de 10 por método detienen el build.
Switch Statements repetidos sobre tipos (bonos, órdenes, acciones)	Decorator para los bonos y Strategy para los órdenes. El único switch del sistema está en FabricaComandosCazador y es exhaustivo sobre el enum: si se agrega un tipo, el compilador obliga a cubrirlo.
Duplicated Code	Una sola clase de parámetros de búsqueda para la web y la API (ParametrosBusqueda), datos de prueba centralizados (Ejemplos) y CPD de PMD con un mínimo de 100 tokens en el build.
Long Parameter List y Primitive Obsession	Builder para CocinaFantasma, records como ContextoCaza, DatosCocina y NuevaResena, y constantes con nombre (PUNTOS_POR_RESENA, MAXIMO_CAZAS_POR_DIA, VENTANA_PARA_DESHACER) en lugar de números sueltos.

Sección 2 — Casos de Uso de Mayor Impacto

Identificar los flujos de negocio que más influyen en las decisiones de diseño. La selección debe justificarse por criterios técnicos: volumen, complejidad, criticidad del negocio o número de componentes involucrados.

Requisito mínimo: Mínimo 8 casos de uso, máximo 10. Todos deben ser de alto impacto.

Completar una ficha por cada caso de uso seleccionado:

2.1 Caso de Uso 1

Nombre del CU	CU-01 Cazar una cocina
Actor principal	Cazador
Criterio de selección como 'alto impacto'	Es el flujo central del negocio y el de más escrituras. Toca más componentes que cualquier otro: controlador, fábrica, invocador, receptor, cadena de

	decoradores, cuatro repositorios y tres observadores. Concentra las reglas críticas (horario, límite diario, caza repetida).
Precondición	Sesión de cazador iniciada; la cocina está activa y abierta; el cazador no la ha cazado y lleva menos de 5 cazas hoy.
Flujo principal	1. El cazador abre la ficha de la cocina y pulsa «Cazar esta cocina». 2. CocinaControlador pide a FabricaComandos un DescubrirCocinaComando y lo entrega a HistorialAcciones. 3. RegistroCaza valida las reglas y arma el ContextoCaza (rareza, primer cazador, días de racha, hora). 4. La cadena de decoradores calcula el puntaje con su desglose. 5. Se guardan el Descubrimiento y la AccionRegistrada del historial. 6. NotificadorProgreso avisa a los observadores de retos, insignias y nivel. 7. Se muestra la comanda con el desglose de puntos y las novedades.
Flujo alternativo (si aplica)	3a. Cocina cerrada, inactiva, ya cazada o límite diario alcanzado: se vuelve a la ficha con el motivo y no se guarda nada.
Postcondición	Caza registrada con sus puntos; la acción se puede deshacer durante 15 minutos; retos, insignias y nivel sincronizados.
Patrón de diseño relacionado y justificación	Command (con Decorator y Observer). Encapsular la caza como objeto permite al invocador registrarla y deshacerla igual que las demás acciones; el Decorator separa cada bono y el Observer desacopla las consecuencias.

2.2 Caso de Uso 2

Nombre del CU	CU-02 Deshacer la última acción
Actor principal	Cazador
Criterio de selección como 'alto impacto'	Es el caso más complejo: debe revertir puntos, reseñas, retos, insignias y nivel de forma consistente. Es la funcionalidad que justifica el patrón asignado e involucra historial, fábrica, los tres receptores y los observadores.
Precondición	Sesión de cazador; existe una acción suya no deshecha realizada hace 15 minutos o menos.
Flujo principal	1. El cazador pulsa «Deshacer» en la barra de última acción. 2. HistorialAcciones toma la última acción no deshecha (orden LIFO). 3. Verifica la ventana de 15 minutos con el Clock inyectado. 4. FabricaComandos.paraDeshacer reconstruye el comando del mismo tipo. 5. El comando ejecuta deshacer() delegando en su receptor. 6. La acción se marca como deshecha y los observadores recalculan el progreso. 7. Se confirma cuántos puntos se retiraron.
Flujo alternativo (si aplica)	2a. No hay acciones: «No tienes acciones pendientes por deshacer». 3a. Pasaron más de 15 minutos: se informa y no cambia nada.
Postcondición	El estado queda igual que antes de la acción; el historial la muestra tachada.
Patrón de diseño relacionado y justificación	Command y Factory Method. El historial persistido guarda lo mínimo para reconstruir el comando; la fábrica crea el comando adecuado según el tipo, sin que el invocador conozca las clases concretas.

2.3 Caso de Uso 3

Nombre del CU	CU-03 Publicar una reseña
Actor principal	Cazador
Criterio de selección como 'alto impacto'	Genera el contenido público que alimenta la calificación promedio, el orden por calificación y la insignia de crítico. Tiene reglas propias (una reseña por caza,

	comentario de 300 caracteres) y debe poder deshacerse.
Precondición	La cocina fue cazada por el cazador y aún no tiene reseña suya.
Flujo principal	1. El cazador elige de 1 a 5 estrellas y escribe un comentario opcional. 2. Bean Validation revisa el formulario. 3. HistorialAcciones ejecuta un PublicarResenaComando. 4. RegistroResenas busca el descubrimiento y llama a Descubrimiento.resenar(). 5. Se suman 20 puntos y se notifica a los observadores. 6. La reseña aparece en la ficha con la etiqueta «Tu reseña».
Flujo alternativo (si aplica)	2a. Datos inválidos: el error se muestra junto al campo. 4a. Sin caza previa o reseña repetida: mensaje de la regla incumplida.
Postcondición	Reseña asociada por composición al descubrimiento; promedio y conteo actualizados.
Patrón de diseño relacionado y justificación	Command. Publicar reseña comparte invocador, historial y ventana de deshacer con las otras acciones.

2.4 Caso de Uso 4

Nombre del CU	CU-04 Explorar cocinas cercanas
Actor principal	Visitante o cazador
Criterio de selección como 'alto impacto'	Es el caso de mayor volumen de lectura y la puerta de entrada al juego. Combina búsqueda sin tildes, filtros, distancia Haversine, horario que cruza la medianoche y estadísticas; la API REST reutiliza el mismo flujo.
Precondición	Ninguna: es público.
Flujo principal	1. El usuario abre /explorar. 2. Escribe un texto, elige categoría, «solo abiertas» y el orden; opcionalmente comparte su ubicación. 3. ServicioCatalogo arma una ficha por cocina con estadísticas y distancia. 4. FiltroCocinas descarta las que no cumplen. 5. SelectorCriterioOrden entrega la estrategia elegida y esta ordena las fichas. 6. La lista y el mapa muestran el mismo resultado.
Flujo alternativo (si aplica)	2a. Ubicación denegada: las distancias se miden desde el Parque Santander. 5a. Sin resultados: mensaje y botón para ver todas.
Postcondición	No modifica datos.
Patrón de diseño relacionado y justificación	Strategy. Cada orden es un algoritmo intercambiable y el selector lo elige en tiempo de ejecución sin condicionales.

2.5 Caso de Uso 5

Nombre del CU	CU-05 Unirse a un reto y completarlo
Actor principal	Cazador
Criterio de selección como 'alto impacto'	Introduce reglas temporales (vigencia, cazas entre la inscripción y el cierre) y recompensas que afectan puntos, nivel e insignias. Exige consistencia entre módulos incluso al deshacer.
Precondición	Reto vigente en el que el cazador no está inscrito.
Flujo principal	1. En /retos el cazador pulsa «Unirme al reto». 2. HistorialAcciones ejecuta UnirseARetoComando y RegistroParticipaciones lo inscribe. 3. Con cada caza posterior, SincronizadorRetos cuenta las cocinas dentro de la

	ventana del reto. 4. Al llegar a la meta marca el reto como completado y suma la recompensa. 5. El cazador ve la novedad «Completaste el reto…».
Flujo alternativo (si aplica)	1a. Reto no vigente o inscripción repetida: mensaje. 3a. Si se deshace la caza que completó el reto, vuelve a pendiente.
Postcondición	Participación registrada; recompensa sumada solo mientras la meta se cumpla.
Patrón de diseño relacionado y justificación	Observer (con Command). El reto reacciona al progreso sin que los comandos sepan que existen los retos.

2.6 Caso de Uso 6

Nombre del CU	CU-06 Otorgar insignias y nivel automáticamente
Actor principal	Sistema (tras cualquier acción)
Criterio de selección como 'alto impacto'	No tiene pantalla propia pero atraviesa todo: debe quedar correcto después de cada acción y de cada deshacer. Evalúa nueve reglas y cinco niveles derivados de los puntos.
Precondición	El cazador ejecutó o deshizo una acción.
Flujo principal	1. HistorialAcciones llama a notificar(cazadorId, puntosAntes). 2. SincronizadorRetos (orden 1) actualiza retos, porque las recompensas cambian los puntos. 3. SincronizadorInsignias (orden 2) recalcula el ProgresoCazador y otorga o retira insignias. 4. DetectorCambioNivel (orden 3) compara el nivel de antes con el de ahora. 5. Las novedades vuelven al controlador y se muestran al cazador.
Flujo alternativo (si aplica)	Al deshacer, los mismos observadores retiran insignias y bajan de nivel si corresponde.
Postcondición	Las insignias guardadas son exactamente las merecidas; el nivel siempre se deriva de los puntos.
Patrón de diseño relacionado y justificación	Observer. Un sujeto con observadores ordenados con @Order; agregar una reacción nueva no toca el resto.

2.7 Caso de Uso 7

Nombre del CU	CU-07 Consultar el panel del cazador
Actor principal	Cazador
Criterio de selección como 'alto impacto'	Es la vista de lectura más compleja: reúne datos de cinco subsistemas (cuenta, progreso, insignias, retos e historial).
Precondición	Sesión de cazador.
Flujo principal	1. El cazador abre /perfil. 2. PerfilControlador llama a PanelCazadorFacade.panelDe(cazadorId). 3. La fachada consulta cuentas, progreso, insignias, retos e historial. 4. Arma un PanelCazador inmutable. 5. La vista muestra puntos, nivel y avance, cifras, insignias, retos, cazas y actividad con opción de deshacer.
Flujo alternativo (si aplica)	Sin cazas: estado vacío con un botón para buscar cocinas abiertas.
Postcondición	No modifica datos.
Patrón de diseño relacionado y justificación	Facade. El controlador depende de una sola clase y no de cinco servicios.

2.8 Caso de Uso 8

Nombre del CU	CU-08 Consultar el ranking
Actor principal	Visitante, cazador o cliente de la API
Criterio de selección como 'alto impacto'	Es público y se expone también por la API. Debe usar la misma fórmula de puntos que el panel y resolver empates.
Precondición	Ninguna.
Flujo principal	1. El usuario abre /ranking o llama GET /api/v1/ranking. 2. ServicioRanking obtiene la actividad agregada y las recompensas con consultas de proyección. 3. PuntosCazador.total() calcula el puntaje de cada cazador. 4. Ordena por puntos y cazas; los empates comparten puesto. 5. Muestra la tabla o el JSON.
Flujo alternativo (si aplica)	Nadie tiene puntos: mensaje invitando a cazar. Límite de la API fuera de rango: se acota entre 1 y 50.
Postcondición	No modifica datos.
Patrón de diseño relacionado y justificación	Ninguno de GoF a propósito: es una consulta sin variaciones de comportamiento. Se usa el Repository de Spring Data con proyecciones; aplicar Strategy aquí sería sobreingeniería.

2.9 Caso de Uso 9

Nombre del CU	CU-09 Gestionar el catálogo de cocinas
Actor principal	Administrador
Criterio de selección como 'alto impacto'	Todos los demás casos dependen de la calidad de estos datos. Tiene la validación más exigente (coordenadas, horario que cruza la medianoche, canales) y una regla de integridad: no borrar cocinas con cazas.
Precondición	Sesión de administrador.
Flujo principal	1. El administrador abre /admin/cocinas y pulsa «Registrar cocina». 2. Completa el formulario. 3. Bean Validation revisa los datos. 4. AdministracionCocinas verifica que el nombre no exista y construye la cocina con CocinaFantasma.Builder. 5. Se guarda y vuelve a la lista. Editar usa actualizar(Builder) y conserva la fecha de alta y el estado.
Flujo alternativo (si aplica)	4a. Nombre repetido: error en el campo. Eliminar una cocina con cazas: se rechaza y se sugiere desactivarla.
Postcondición	Catálogo actualizado; una cocina inactiva deja de aparecer pero conserva sus cazas y reseñas.
Patrón de diseño relacionado y justificación	Builder. La cocina tiene diez datos con reglas cruzadas; el Builder evita un constructor de muchos parámetros y concentra la validación en un solo punto.

2.10 Caso de Uso 10

Nombre del CU	CU-10 Registrarse e iniciar sesión
Actor principal	Visitante
Criterio de selección como 'alto	Es la base de la seguridad: el rol decide qué puede hacer cada usuario. Exige

impacto'	correo y alias únicos y claves cifradas.
Precondición	No tener sesión iniciada.
Flujo principal	1. El visitante abre /registro y escribe nombre, alias, correo, clave y confirmación. 2. Bean Validation revisa formato y coincidencia de claves. 3. ServicioCuentas normaliza el correo, valida el alias, verifica unicidad y cifra la clave con BCrypt. 4. Redirige a /ingresar con un mensaje de confirmación. 5. Spring Security autentica y RedireccionPorRol envía al cazador a /explorar y al administrador a /admin/cocinas.
Flujo alternativo (si aplica)	3a. Correo o alias en uso: error en el campo. 5a. Clave incorrecta: mensaje en la pantalla de ingreso.
Postcondición	Cuenta creada y sesión iniciada con su rol.
Patrón de diseño relacionado y justificación	Template Method. RedireccionPorRol redefine solo el paso determineTargetUrl() del algoritmo de Spring Security.

Sección 3 — Modelo de Clases

El modelo de clases debe reflejar el diseño OO completo con relaciones y responsabilidades bien definidas. Es la base sobre la que se aplican los patrones de la Sección 4.

3.1 Diagrama UML de Clases

Insertar el diagrama UML de clases a continuación. El diagrama debe incluir visibilidades (+, -, #), tipos de atributo, firmas de método y tipos de relación (herencia, composición, agregación, dependencia) correctamente diferenciados.

Requisito mínimo: Mínimo 8 clases. Señalar con [P] las clases candidatas a recibir patrones de diseño.

El modelo tiene más de cuarenta clases, por eso se presenta en siete vistas del mismo diagrama, una por página horizontal: dos del dominio y una por cada patrón. Notación: + público, - privado, # protegido, ~ de paquete, subrayado estático, cursiva abstracto. Herencia con triángulo y línea continua, realización con triángulo y línea punteada, composición con rombo relleno, agregación con rombo vacío, asociación con flecha continua y dependencia con flecha punteada. Las clases resaltadas y marcadas con [P] participan en un patrón (ámbar: Builder, Command y Decorator; lila: Factory Method y Strategy; verde: Observer y Facade). Las versiones SVG, que se pueden ampliar sin perder nitidez, están en docs/diagramas del repositorio.

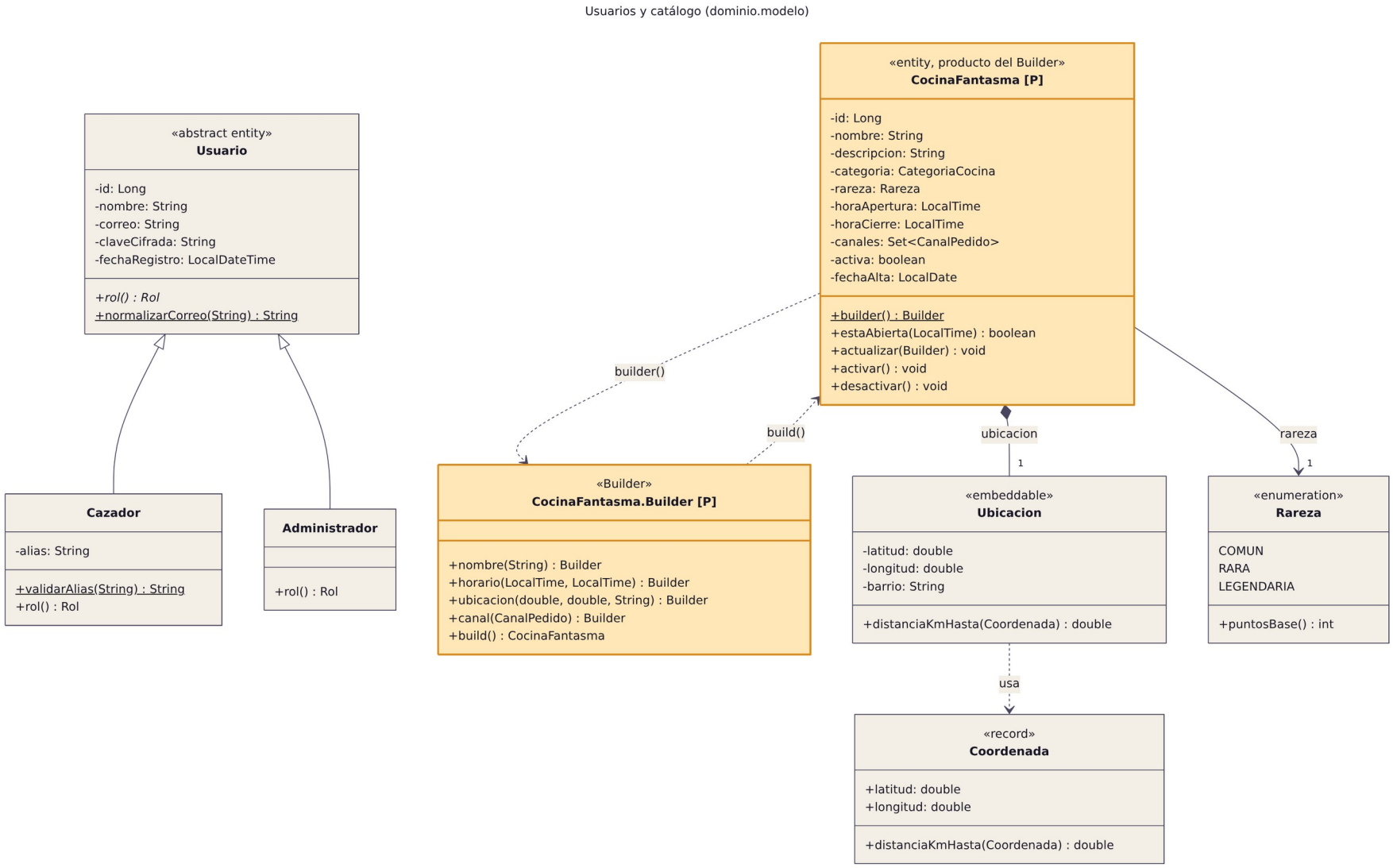


Figura 1. Usuarios y catálogo: herencia de Usuario, composición con Ubicacion y el Builder de CocinaFantasma.

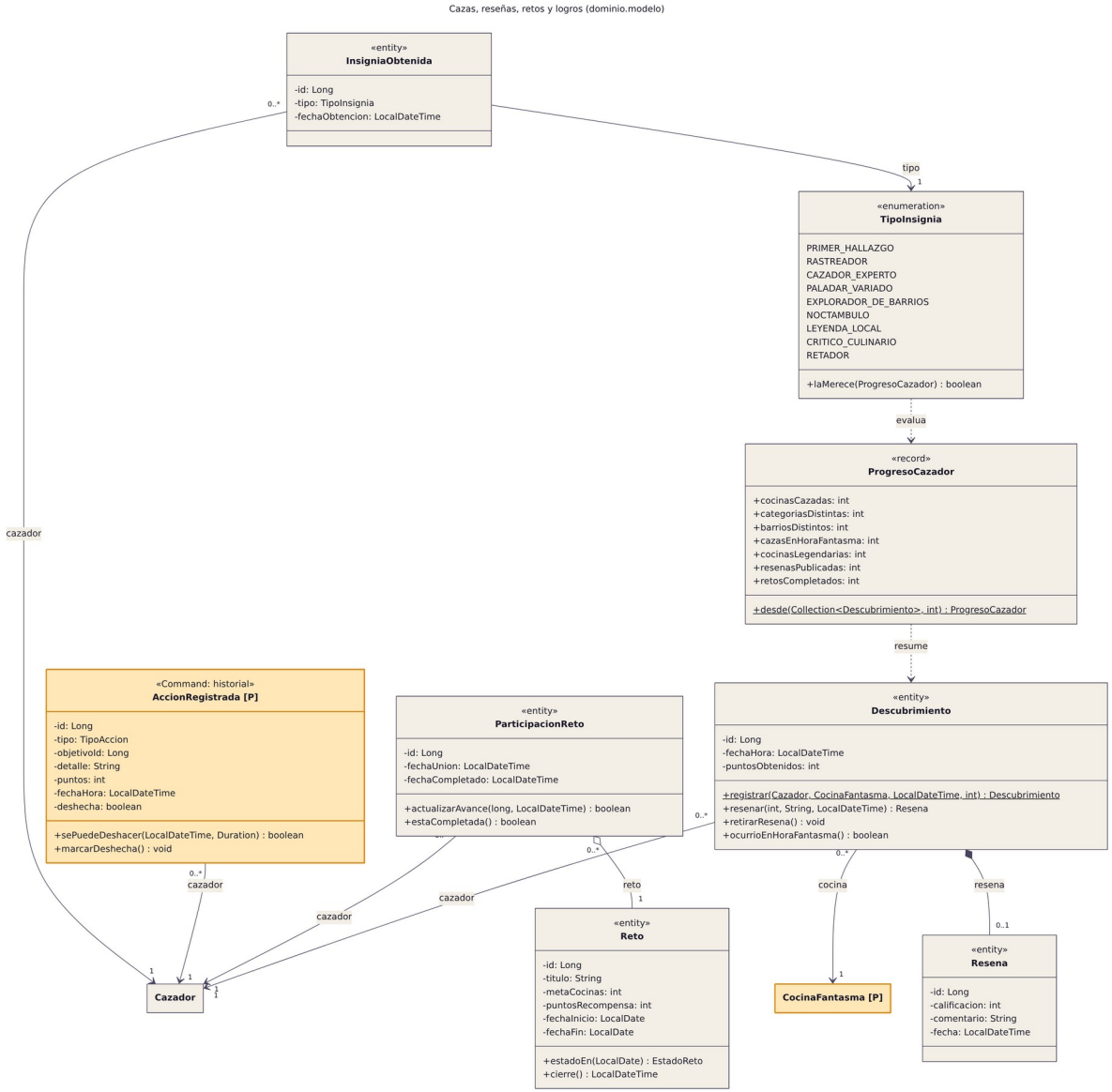


Figura 2. Cazas, reseñas, retos y logros: asociaciones con multiplicidad, composición Descubrimiento y Resena, agregación con Reto.

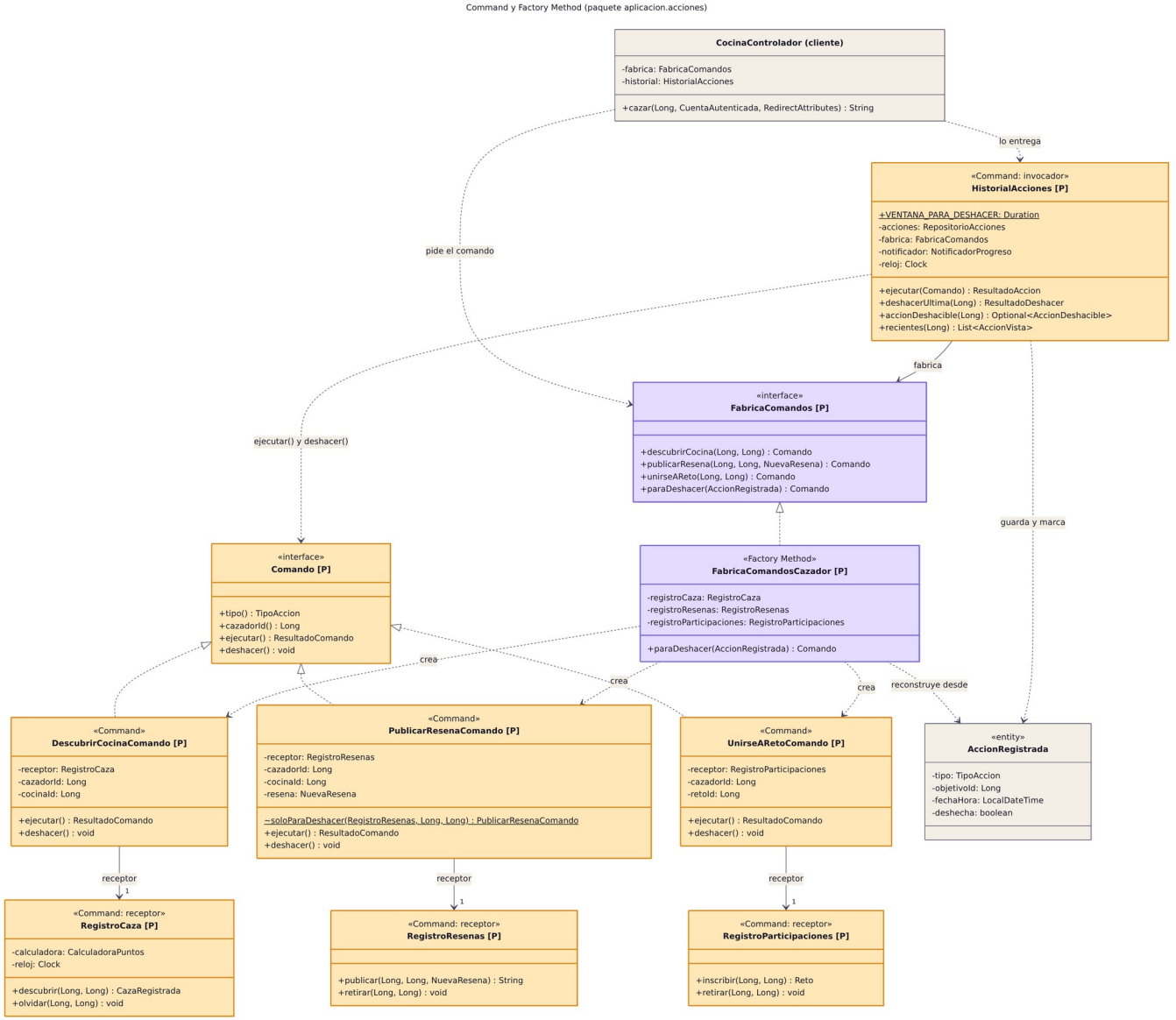


Figura 3. Command y Factory Method: cliente, invocador, comandos concretos, receptores y fábrica.

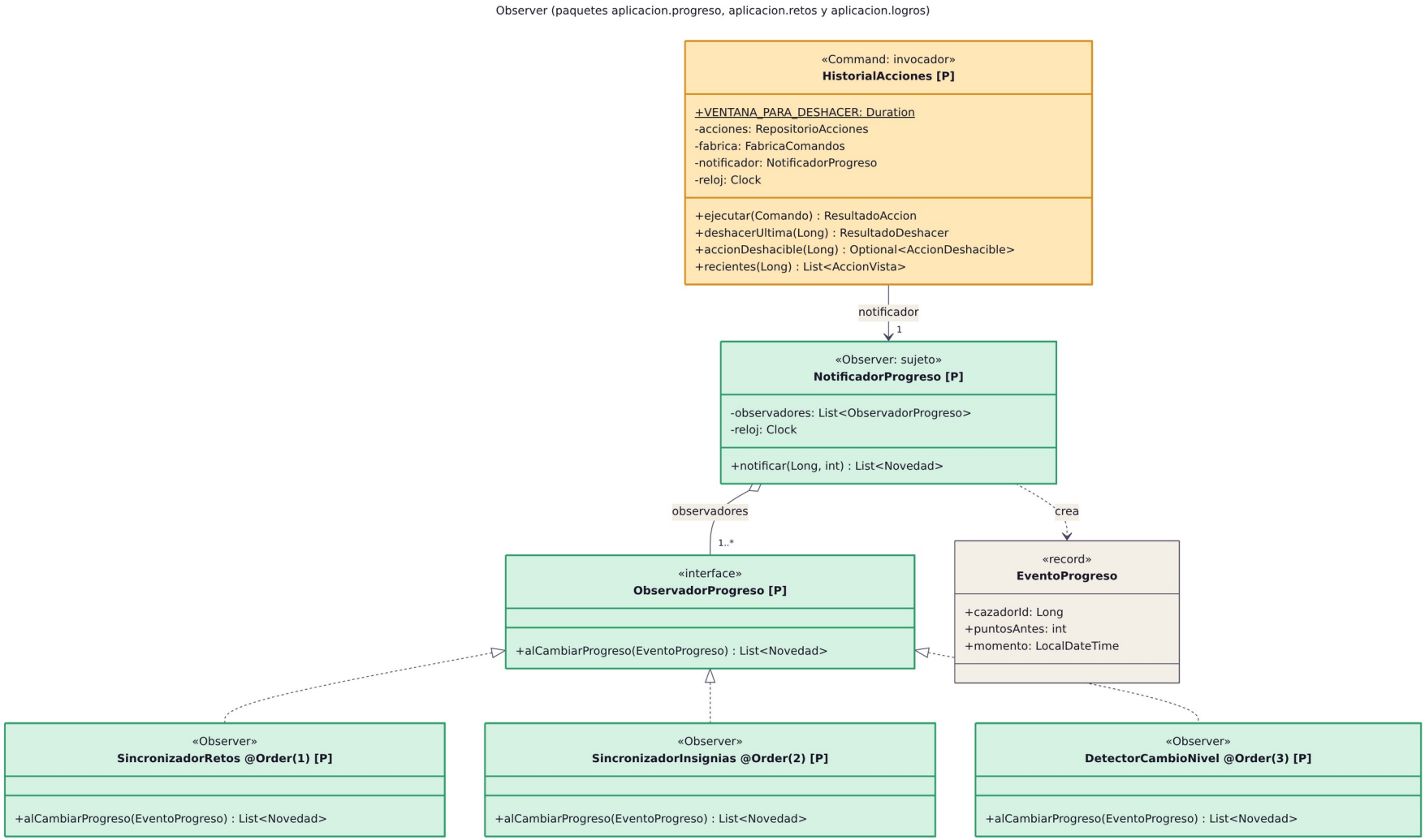


Figura 4. Observer: sujeto NotificadorProgreso y observadores ordenados.

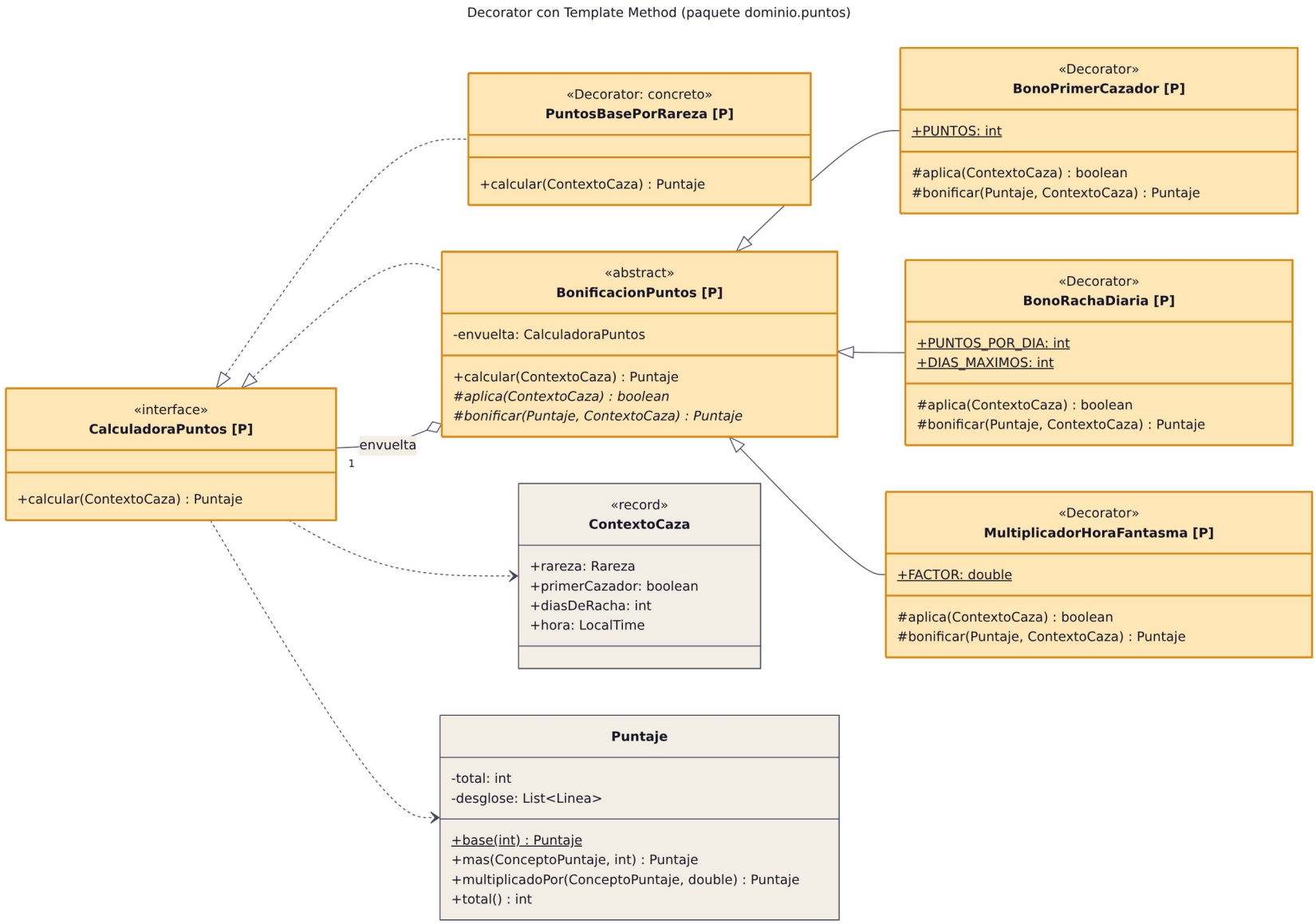


Figura 5. Decorator con Template Method en el cálculo de puntos.

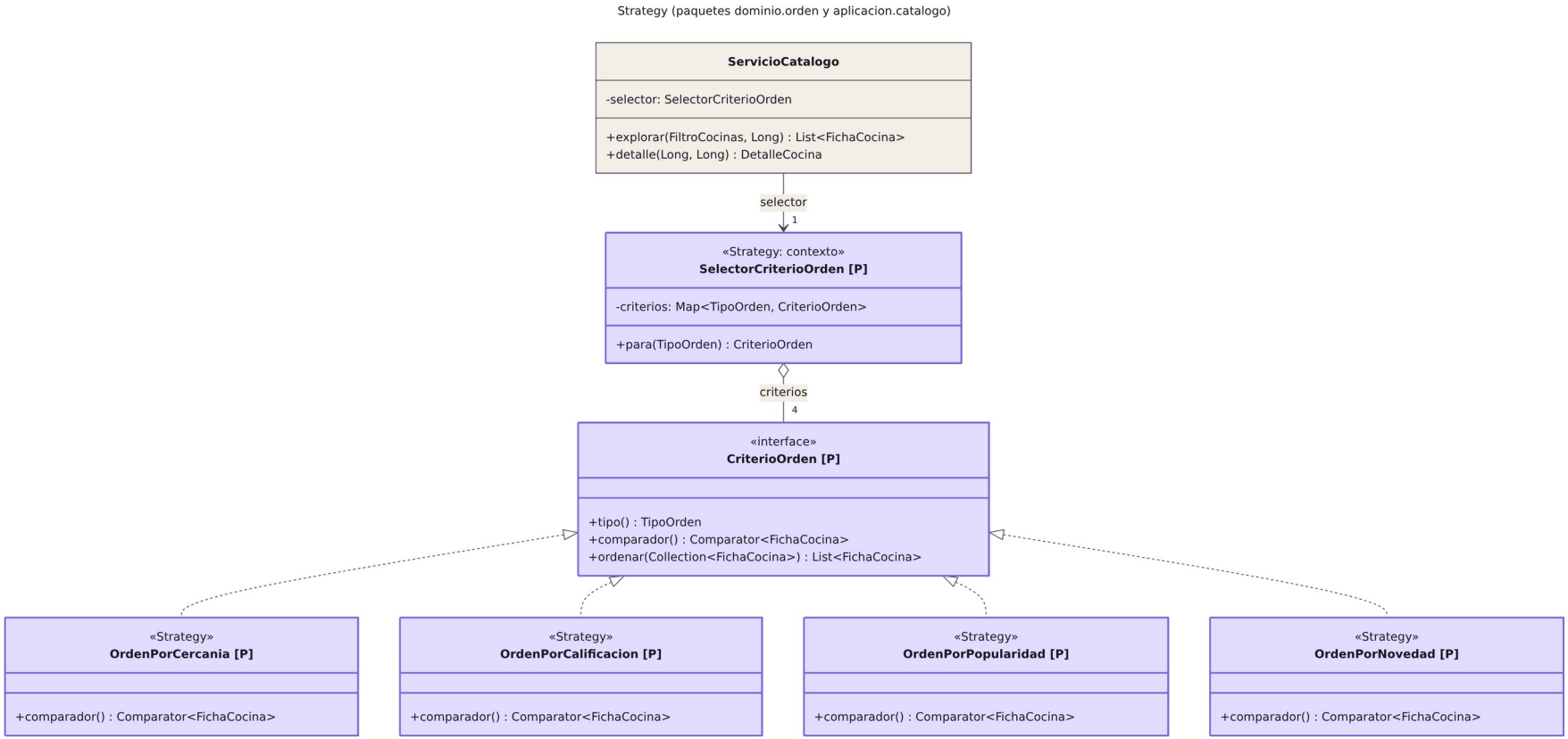


Figura 6. Strategy: criterios de orden del explorador y su selector.

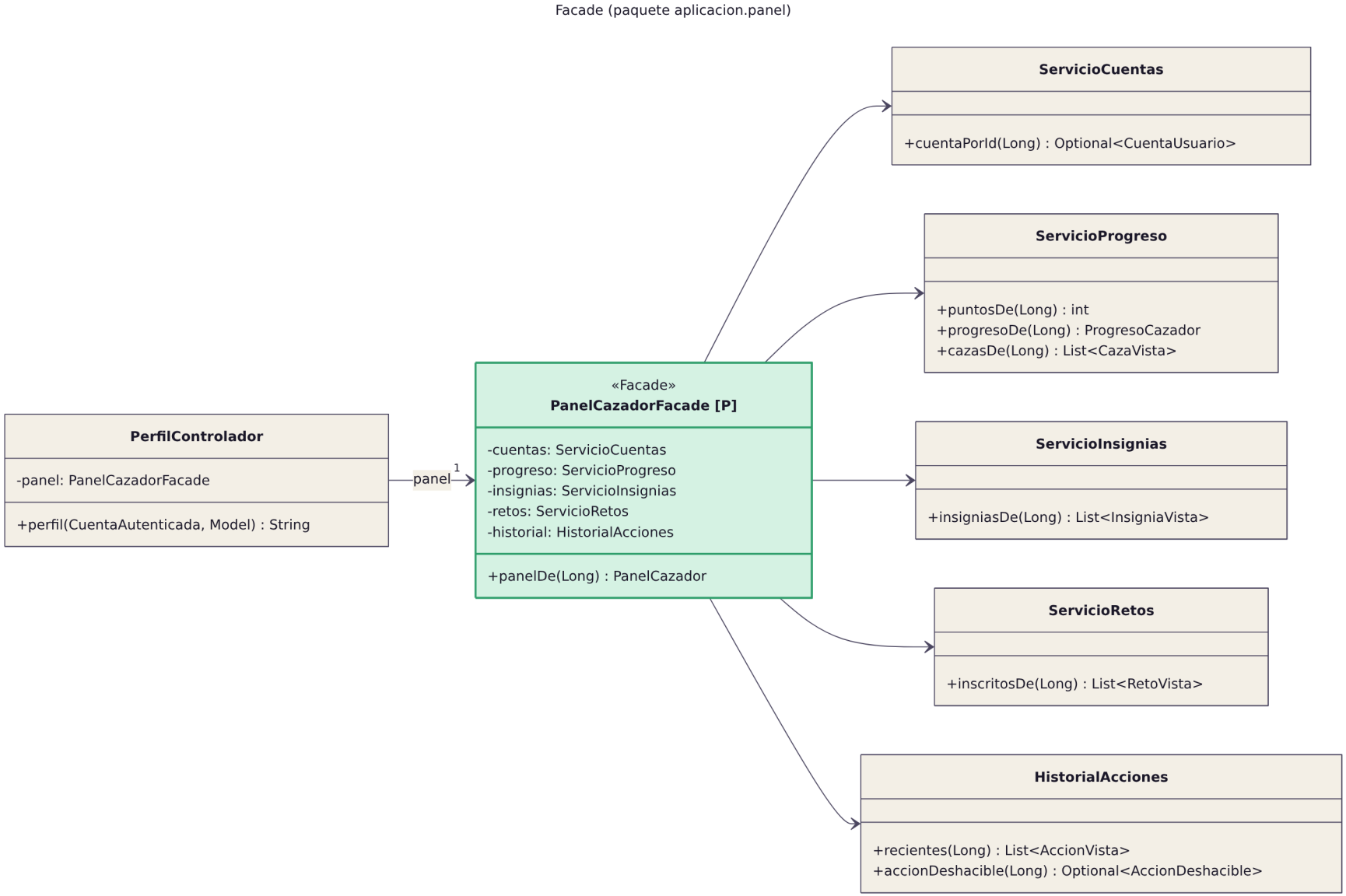


Figura 7. Facade: el panel del cazador oculta cinco subsistemas.

3.2 Tabla de Responsabilidades

Por cada clase principal, definir su única responsabilidad (SRP) y la capa o paquete al que pertenece.

Clase	Responsabilidad única (SRP)	Paquete / Capa	Candidata a patrón [P]
CocinaFantasma	Mantener los datos de una cocina y decidir si está abierta a una hora dada.	dominio.modelo / Dominio	Sí (Builder)
CocinaFantasma.Builder	Reunir y validar los datos para crear o actualizar una cocina.	dominio.modelo / Dominio	Sí (Builder)
Descubrimiento	Registrar una caza y ser dueño de su reseña opcional.	dominio.modelo / Dominio	No
Reto / ParticipacionReto	Definir una misión temporal / llevar el avance de un cazador en ella.	dominio.modelo / Dominio	No
AccionRegistrada	Guardar lo mínimo de una acción para mostrarla y deshacerla.	dominio.modelo / Dominio	Sí (Command)
TipoInsignia / Nivel	Encapsular la regla de cada insignia / derivar el nivel desde los puntos.	dominio.modelo / Dominio	No
CalculadoraPuntos	Contrato para calcular el puntaje de una caza.	dominio.puntos / Dominio	Sí (Decorator)
BonificacionPuntos y sus 3 subclases	Aplicar un único bono sobre el puntaje que reciben.	dominio.puntos / Dominio	Sí (Decorator)
CriterioOrden y sus 4 implementaciones	Ordenar fichas de cocina según un criterio.	dominio.orden / Dominio	Sí (Strategy)
SelectorCriterioOrden	Entregar la estrategia de orden pedida.	aplicacion.catalogo / Aplicación	Sí (Strategy)
ServicioCatalogo	Armar fichas de cocina con estadísticas, filtro y orden.	aplicacion.catalogo / Aplicación	No
Comando y sus 3 comandos	Encapsular una acción del cazador con su forma de deshacerla.	aplicacion.acciones / Aplicación	Sí (Command)
HistorialAcciones	Ejecutar comandos, registrarlos y deshacer el último.	aplicacion.acciones / Aplicación	Sí (Command)
FabricaComandosCazador	Crear comandos nuevos o reconstruirlos desde el historial.	aplicacion.acciones / Aplicación	Sí (Factory Method)
RegistroCaza / RegistroResenas / RegistroParticipaciones	Hacer el trabajo real de cada acción y revertirlo (receptores).	aplicacion.acciones y retos / Aplicación	Sí (Command)
NotificadorProgreso	Avisar a los observadores cuando cambia el progreso.	aplicacion.progreso / Aplicación	Sí (Observer)
SincronizadorRetos / SincronizadorInsignias / DetectorCambioNivel	Reaccionar al progreso: retos, insignias o nivel.	aplicacion.retos y logros / Aplicación	Sí (Observer)
PanelCazadorFacade	Reunir en un objeto los datos del panel del	aplicacion.panel /	Sí

Clase	Responsabilidad única (SRP)	Paquete / Capa	Candidata a patrón [P]
	cazador.	Aplicación	(Facade)
ServicioRanking	Calcular la tabla de posiciones con la fórmula común.	aplicacion.ranking / Aplicación	No
ServicioCuentas	Registrar cazadores y consultar cuentas.	aplicacion.cuentas / Aplicación	No
Repositorio* (8 interfaces)	Acceder a los datos de un agregado.	persistencia / Persistencia	No
CocinaControlador y demás controladores	Traducir peticiones HTTP y elegir la vista.	presentacion.web / Presentación	No
RedireccionPorRol	Decidir a dónde va cada rol tras iniciar sesión.	seguridad / Transversal	Sí (Template Method)
ConfiguracionDominio	Componer la cadena de decoradores, las estrategias y el reloj.	configuracion / Transversal	No

Sección 4 — Propuesta de Aplicación de Patrones de Diseño

Esta sección evalúa el criterio de selección de patrones: saber cuándo y por qué aplicar cada uno, y también cuándo NO aplicarlo. La justificación técnica tiene más peso que la cantidad de patrones propuestos.

> **Requisito mínimo:** Al menos 1 patrón por categoría (creacional, estructural, comportamiento). Si se propone un patrón, debe justificarse por qué ese y no otro similar.

4.1 Patrones Creacionales

Patrón propuesto	Clase(s) donde aplica	Problema que resuelve	¿Por qué este y no otro?
Builder	CocinaFantasma.Builder, CocinaFantasma.actualizar(Builder), DatosCocina.aplicarA()	Una cocina tiene diez datos con reglas cruzadas (horario distinto, al menos un canal, coordenadas válidas). Un constructor de diez parámetros es ilegible y fácil de usar mal.	Factory Method no resuelve el problema de muchos parámetros opcionales. El Builder además se reutiliza para editar: actualizar() recibe un Builder y valida igual que al crear.
Factory Method	FabricaComandos, FabricaComandosCazador (paraDeshacer, descubrirCocina…)	En una aplicación web el objeto comando muere con la petición. Para deshacer hay que recrear el comando correcto a partir de la AccionRegistrada, sin que el invocador conozca las clases concretas.	Abstract Factory crea familias de productos y aquí hay un solo producto (Comando). Un constructor directo acoplaría el controlador y el historial a cada comando concreto.

4.2 Patrones Estructurales

Patrón propuesto	Clase(s) donde aplica	Problema que resuelve	¿Por qué este y no otro?
Decorator	CalculadoraPuntos, PuntosBasePorRareza,	Los puntos de una caza se forman con bonos que se combinan y	Un if/else gigante crecería con cada bono. Chain of Responsibility corta

Patrón propuesto	Clase(s) donde aplica	Problema que resuelve	¿Por qué este y no otro?
	BonificacionPuntos, BonoPrimerCazador, BonoRachaDiaria, MultiplicadorHoraFantasma	cuyo orden importa: el multiplicador de la hora fantasma debe ir por fuera para multiplicar también los bonos fijos.	la cadena cuando alguien atiende, y aquí todos los bonos aportan. Strategy elige uno solo, no los combina.
Facade	PanelCazadorFacade	El panel necesita datos de cinco servicios. Sin fachada el controlador dependería de todos y repetiría la orquestación.	Adapter traduce una interfaz incompatible y aquí no la hay. Mediator coordina objetos que se hablan entre sí; los servicios del panel no se comunican.

4.3 Patrones de Comportamiento

Patrón propuesto	Clase(s) donde aplica	Problema que resuelve	¿Por qué este y no otro?
Command (patrón asignado)	Comando, DescubrirCocinaComando, PublicarResenaComando, UnirseARetoComando, HistorialAcciones (invocador), RegistroCaza, RegistroResenas, RegistroParticipaciones (receptores)	Las acciones del cazador deben ejecutarse, quedar en un historial y poder deshacerse con las mismas reglas (pila LIFO y ventana de 15 minutos) sin que el invocador conozca cada acción.	Memento guarda instantáneas completas del estado; aquí basta con saber cómo revertir cada acción. Un servicio con un método por acción no tendría historial uniforme ni un único punto para deshacer.
Strategy	CriterioOrden, OrdenPorCercania, OrdenPorCalificacion, OrdenPorPopularidad, OrdenPorNovedad, SelectorCriterioOrden	El explorador ordena la misma lista de cuatro maneras elegidas por el usuario en cada petición.	Template Method fijaría el algoritmo por herencia y no permite cambiarlo en tiempo de ejecución. State no aplica: el orden no es un estado del objeto, es una elección del usuario.
Observer	NotificadorProgreso, ObservadorProgreso, SincronizadorRetos, SincronizadorInsignias, DetectorCambioNivel	Después de cada acción o deshacer hay que actualizar retos, insignias y nivel, y los comandos no deben conocer esas reacciones.	Los eventos de Spring no garantizan el orden por defecto ni devuelven resultados; aquí el orden importa (retos antes que nivel) y se necesitan las novedades para mostrarlas.
Template Method (complementario)	BonificacionPuntos (calcular final, aplica y bonificar abstractos); RedireccionPorRol	Todos los bonos siguen el mismo esqueleto: calcular lo envuelto y, si aplica, bonificar. Tras iniciar sesión solo cambia el destino.	Fija el esqueleto y protege el contrato del Decorator (calcular es final). Redefinir un paso es más simple que reescribir el manejador de Spring Security.

Patrones evaluados y descartados. Cada descarte se ancla en un principio de la guía de la unidad 12 (YAGNI, KISS, regla de tres o prueba de explicabilidad) y deja escrita la señal que haría reconsiderarlo. Chain of Responsibility para validar la caza: son cuatro reglas fijas que se leen en unas doce líneas de cláusulas de guarda; la única justificación posible era «por si cambian», que no pasa la prueba de explicabilidad (se reconsidera si el administrador necesita activar o desactivar reglas). State para el nivel: el nivel se deriva de los puntos y no cambia lo que el cazador puede hacer (KISS; se reconsidera si un nivel desbloquea acciones). Memento para deshacer: cada comando ya sabe revertir lo que hizo (KISS; se reconsidera con una acción sin reversa natural, como editar una reseña). Abstract Factory de comandos por rol: solo existe la familia del

cazador (regla de tres). Singleton manual: Spring ya administra una instancia por bean y uno estático impediría inyectar dobles. Puertos hexagonales, rehacer, eventos asíncronos y CQRS: nadie los necesita hoy (YAGNI y KISS). La tabla completa, con la señal de cada uno, está en la sección «Qué se descartó y por qué» del README.

4.4 Antipatrones Identificados y Descartados

Identificar antipatrones que podrían surgir de manera natural en el diseño del sistema y la estrategia activa para prevenirlos.

Requisito mínimo: Mínimo 2 antipatrones con estrategia de mitigación concreta.

Antipatrón en riesgo	Dónde podría aparecer	Estrategia de prevención
God Object	Un ServicioCazador que cace, reseñe, deshaga, calcule puntos y arme el panel.	Separación en receptores, invocador, observadores y fachada. Regla GodClass y complejidad máxima de 10 en PMD.
Golden Hammer	Aplicar patrones por aplicarlos: Strategy en el ranking, Singleton manual, CQRS o Event Sourcing.	Cada patrón debe resolver un problema real registrado en la tabla anterior; los descartados quedan justificados en 4.3 y en los ADR.
Modelo de dominio anémico	Entidades con solo getters y toda la lógica en servicios.	Las reglas viven en el dominio: estaAbierta(), resenar(), actualizarAvance(), sePuedeDeshacer(), Nivel.paraPuntos(). El dominio se prueba sin Spring.
Copy-paste programming	Filtros repetidos en la web y la API; datos de prueba repetidos.	ParametrosBusqueda compartido, clase Ejemplos para pruebas y CPD con mínimo de 100 tokens en el build.
Lazy loading en la vista (Open Session in View)	Plantillas que navegan entidades y lanzan consultas.	spring.jpa.open-in-view=false y vistas que solo reciben records; ArchUnit prohíbe que la presentación use entidades.

Sección 5 — Arquitectura del Sistema

Justificar la arquitectura elegida y demostrar su coherencia con los patrones de diseño propuestos. Elegir con criterio: la arquitectura debe ajustarse al tamaño y complejidad real del sistema, no al patrón más sofisticado.

5.1 Patrón(es) Arquitectónico(s) Propuesto(s)

Opciones válidas del curso (elegir al menos 1, combinar si se justifica):

Arquitectura en Capas · MVC con Spring Boot · Arquitectura Hexagonal · Clean Architecture · DDD + CQRS · Event Sourcing

Patrón arquitectónico	Justificación de elección	Limitación asumida
Arquitectura en Capas	Cuatro capas con dependencias en un solo sentido (presentación, aplicación, persistencia y dominio) más seguridad y configuración transversales. Es suficiente para un sistema de un solo despliegue y sin integraciones externas, y ArchUnit la hace verificable.	El dominio usa anotaciones JPA (ADR-003): cambiar a una base no relacional obligaría a tocar las entidades. Se acepta porque no hay otro mecanismo de persistencia previsto.
MVC con Spring Boot	Controladores delgados, vistas Thymeleaf renderizadas en el servidor y un modelo formado por records de vista. Encaja con formularios, sesiones y CSRF sin un frontend aparte.	Las páginas se recargan en cada acción. Se mitiga con el patrón Post/Redirect/Get y pequeñas mejoras en JavaScript.
Hexagonal (descartada)	Evaluada: puertos y adaptadores protegerían el	No hay sistemas externos que

Patrón arquitectónico	Justificación de elección	Limitación asumida
	dominio de la infraestructura.	cambiar; los puertos de persistencia duplicarían los repositorios sin beneficio (YAGNI, ADR-001).

5.2 Diagrama de Arquitectura

Insertar el diagrama de arquitectura (por capas o por componentes) mostrando el flujo de dependencias entre módulos. Cada capa o componente debe tener una responsabilidad definida.

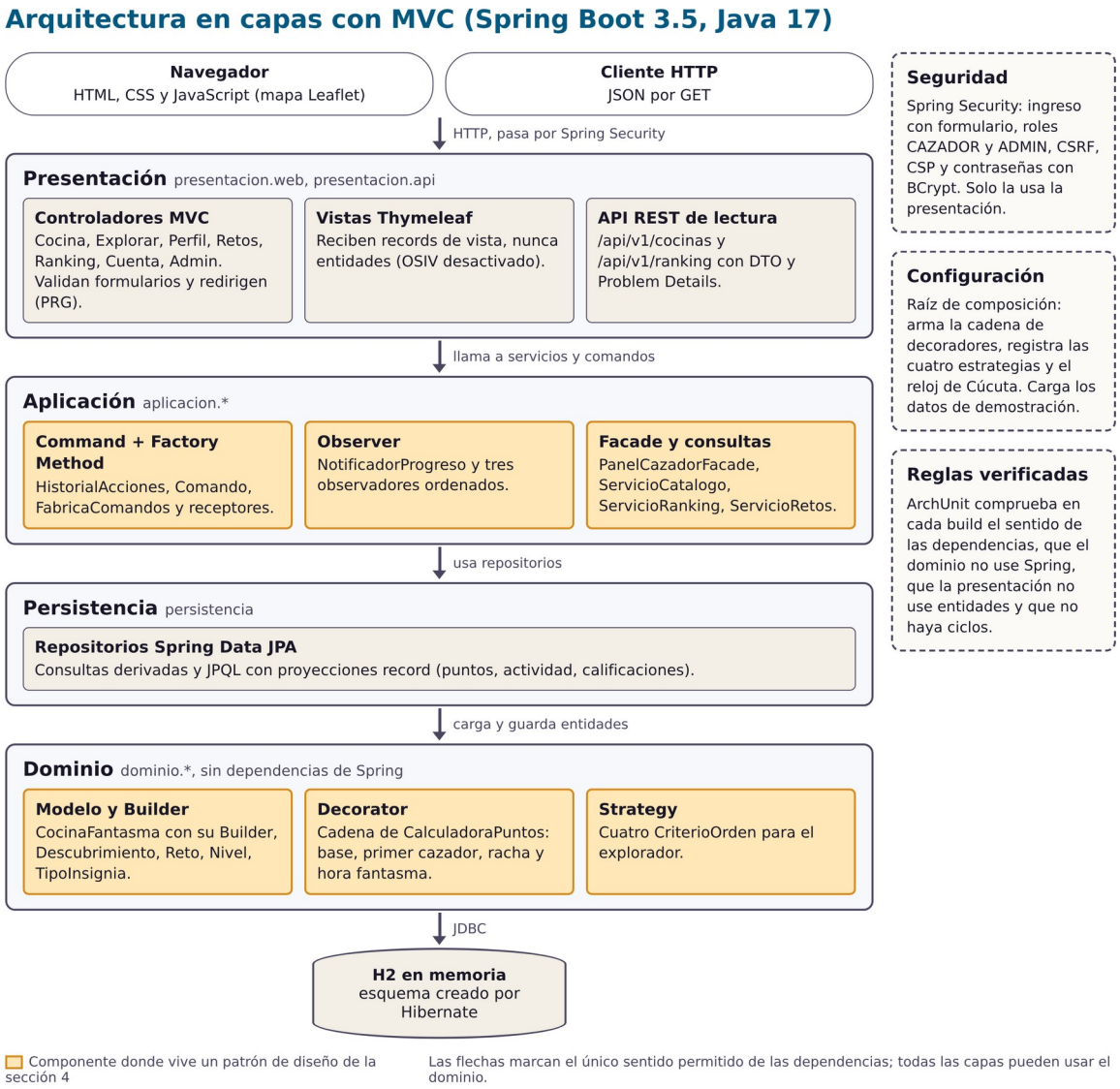


Figura 8. Capas, responsabilidades y sentido de las dependencias.

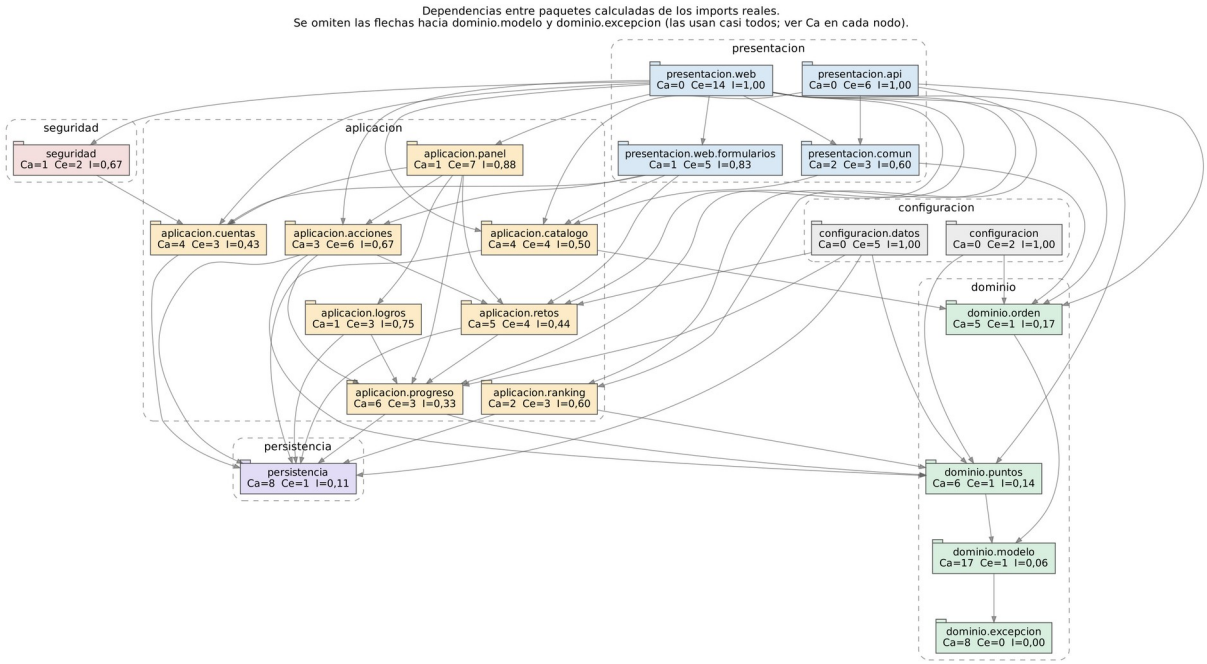


Figura 9. Dependencias entre paquetes calculadas de los imports, con Ca, Ce e inestabilidad (I) de cada uno.

5.3 Validación de Coherencia

Responder las siguientes preguntas de validación:

¿Existe sobreingeniería en la arquitectura propuesta?	No. Se eligió la arquitectura más simple que separa responsabilidades: capas con MVC. No hay microservicios, colas, CQRS ni Event Sourcing porque hay un solo despliegue, una base de datos y lecturas sencillas. No se agregaron puertos de persistencia porque Spring Data ya es una abstracción. Cada patrón responde a un caso de uso concreto de la Sección 2 y pasa la prueba de explicabilidad (se justifica en una oración en la sección «Diseño propuesto» del README); el ranking, que no lo necesita, no usa ninguno. Los límites se verifican con nueve reglas de ArchUnit (ARQ-01 a ARQ-09) que corren en GitHub Actions; para demostrar que sí detienen el build, la rama demo/violacion-arquitectura hace que un controlador use un repositorio directamente y el build falla, mientras que main tiene el revert y queda en verde. El acoplamiento medido sobre los imports reales (Figura 9) muestra 74 dependencias entre 20 paquetes, sin ciclos y siempre hacia paquetes más estables.
¿Cómo se conectan los patrones de diseño (S4) con la arquitectura (S5)?	Al cazar (CU-01): el controlador de la capa de presentación pide el comando a la fábrica (Factory Method) y lo entrega al invocador (Command) en la capa de aplicación; el receptor usa la cadena de decoradores (Decorator) del dominio y los repositorios de persistencia; al final el sujeto (Observer) avisa a los observadores. La configuración arma la cadena de decoradores y registra las estrategias (Strategy) como beans, así ninguna capa crea dependencias concretas. El diagrama de secuencia completo está en docs/diagramas/secuencia-cazar.
¿Qué componentes podrían crecer o cambiar en el futuro, y cómo la arquitectura lo soporta?	Nuevos bonos de puntos (otra subclase de BonificacionPuntos y una línea en ConfiguracionDominio); nuevos órdenes (otro bean CriterioOrden); nuevas acciones deshacibles (otro Comando, y el switch exhaustivo de la fábrica obliga a cubrirlo); nuevas reacciones al progreso (otro ObservadorProgreso con su @Order); una base de datos real (solo cambia la configuración del datasource); una aplicación móvil (consumiría la API REST existente).

Sección 6 — Escenarios de Prueba

Los escenarios de prueba deben derivarse directamente desde los casos de uso (Sección 2) y los patrones propuestos (Sección 4). No son pruebas genéricas: cada prueba debe poder trazarse a una decisión de diseño.

6.1 Pruebas Unitarias

Requisito mínimo: Mínimo 10 escenarios.

Plantilla por escenario de prueba unitaria:

Cada escenario está implementado con el mismo identificador en @DisplayName dentro de src/test/java. Las pruebas unitarias no levantan Spring y usan JUnit 5, AssertJ y Mockito.

PU-001: CalculadoraPuntos.calcular()	
ID:	PU-001
Clase bajo prueba:	CalculadoraPuntos (cadena de decoradores)
Método:	`calcular()`
Precondición / datos de entrada:	Cocina legendaria, primer cazador, 7 días de racha, 11:00 p. m.
Resultado esperado:	525 puntos con desglose en orden: base 250, primer cazador 50, racha 50 (tope de 5 días), hora fantasma 175.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Es lógica pura del dominio; los decoradores reales son el objeto bajo prueba.

PU-002: BonoPrimerCazador.calcular()	
ID:	PU-002
Clase bajo prueba:	BonoPrimerCazador
Método:	`calcular()`
Precondición / datos de entrada:	Un BonoPrimerCazador que envuelve un PuntosBasePorRareza espiado; cocina rara, no es primer cazador.
Resultado esperado:	La calculadora envuelta se llama exactamente una vez y el puntaje queda en 150 sin bono.
Tipo de Test Double:	Spy
Justificación del tipo de Test Double:	Se necesita el comportamiento real de la base y, a la vez, verificar que el decorador delega.

PU-003: RachaDiaria.diasConsecutivosHasta()	
ID:	PU-003
Clase bajo prueba:	RachaDiaria
Método:	`diasConsecutivosHasta()`
Precondición / datos de entrada:	Cazas ayer, dos veces antier y hace 4 días; hoy 10 de noviembre.
Resultado esperado:	2 días: cuenta hasta ayer, ignora repetidos y se corta en el primer hueco.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Función estática sin dependencias.

PU-004: CocinaFantasma.estaAbierta()	
ID:	PU-004
Clase bajo prueba:	CocinaFantasma
Método:	`estaAbierta()`
Precondición / datos de entrada:	Horarios 12:00 a 23:00 y 18:00 a 02:00 con horas en los bordes (prueba parametrizada de 8 casos).
Resultado esperado:	Abierta desde la apertura incluida hasta antes del cierre, también cruzando la medianoche.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Regla del dominio sin colaboradores.

PU-005: CocinaFantasma.Builder.build()	
ID:	PU-005
Clase bajo prueba:	CocinaFantasma.Builder
Método:	`build()`
Precondición / datos de entrada:	Builder sin canales, con apertura igual al cierre, nombre en blanco o sin fecha de alta.
Resultado esperado:	IllegalArgumentException en cada caso; con datos válidos queda activa y con el nombre sin espacios.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Se prueba el producto real del Builder.

PU-006: Nivel.paraPuntos()	
ID:	PU-006
Clase bajo prueba:	Nivel
Método:	`paraPuntos()`
Precondición / datos de entrada:	Puntos en cada límite: 0, 299, 300, 799, 800, 1600, 2999, 3000 y 99999.
Resultado esperado:	Novato, Novato, Explorador, Explorador, Rastreador, Cazador élite, Cazador élite, Leyenda, Leyenda.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Enum con lógica pura.

PU-007: Descubrimiento.resenar()	
ID:	PU-007
Clase bajo prueba:	Descubrimiento
Método:	`resenar()`
Precondición / datos de entrada:	Un descubrimiento que ya tiene una reseña recibe una segunda.
Resultado esperado:	ReglaNegocioException con código resena.duplicada.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	La regla vive en la entidad y no requiere colaboradores.

PU-008: ParticipacionReto.actualizarAvance()	
ID:	PU-008
Clase bajo prueba:	ParticipacionReto
Método:	`actualizarAvance()`
Precondición / datos de entrada:	Reto con meta 4; avance 3, luego 4, luego 5 y después 3.
Resultado esperado:	Solo devuelve true al pasar a completado; conserva la fecha de la primera vez y se revierte al bajar de la meta.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Entidad con estado propio.

PU-009: AccionRegistrada.sePuedeDeshacer()	
ID:	PU-009
Clase bajo prueba:	AccionRegistrada
Método:	`sePuedeDeshacer()`
Precondición / datos de entrada:	Acción de las 9:00 p. m. consultada a los 15 min, a los 15 min y 1 s, y después de deshecha.
Resultado esperado:	true, false y false.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Recibe la hora como parámetro, así que no necesita un reloj.

PU-010: ProgresoCazador y TipoInsignia.desde() y laMerece()	
ID:	PU-010
Clase bajo prueba:	ProgresoCazador y TipoInsignia
Método:	`desde() y laMerece()`
Precondición / datos de entrada:	Dos cazas: una legendaria en hora fantasma con reseña y otra común de día; 1 reto completado.
Resultado esperado:	Progreso (2, 2, 2, 1, 1, 1, 1); merece Primer hallazgo, Noctámbulo, Leyenda local y Retador, y no las demás.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Objetos de valor del dominio.

PU-011: OrdenPorCalificacion.ordenar()	
ID:	PU-011
Clase bajo prueba:	OrdenPorCalificacion
Método:	`ordenar()`
Precondición / datos de entrada:	Tres fichas: dos con 4,5 estrellas (6 y 1 reseñas) y una con 4,0.
Resultado esperado:	Primero la de 4,5 con más reseñas, luego la otra de 4,5 y al final la de 4,0.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Estrategia pura sobre records.

PU-012: RegistroCaza.descubrir()	
ID:	PU-012
Clase bajo prueba:	RegistroCaza
Método:	`descubrir()`
Precondición / datos de entrada:	Cocina que abre de 6:00 a 14:00 consultada a las 9:30 p. m.
Resultado esperado:	ReglaNegocioException caza.cocina-cerrada; la calculadora nunca se usa y no se guarda nada.
Tipo de Test Double:	Stub y Dummy
Justificación del tipo de Test Double:	Los repositorios son Stub (devuelven la cocina); la calculadora es Dummy: se pasa porque el constructor la pide y se verifica que no se toca. El reloj es un Fake (Clock.fixed).

PU-013: RegistroCaza.descubrir()	
ID:	PU-013
Clase bajo prueba:	RegistroCaza
Método:	`descubrir()`
Precondición / datos de entrada:	El repositorio indica 5 cazas hoy para el cazador.
Resultado esperado:	ReglaNegocioException caza.limite-diario.
Tipo de Test Double:	Stub
Justificación del tipo de Test Double:	Solo hace falta controlar la respuesta de cazasDelDia(); no se verifica ninguna interacción.

PU-014: HistorialAcciones.ejecutar()	
ID:	PU-014
Clase bajo prueba:	HistorialAcciones
Método:	`ejecutar()`
Precondición / datos de entrada:	Comando que devuelve 150 puntos; el cazador tenía 280 antes.
Resultado esperado:	En orden: ejecuta el comando, guarda la AccionRegistrada con 150 puntos y la hora del reloj, y notifica con 280.
Tipo de Test Double:	Mock
Justificación del tipo de Test Double:	Lo que importa es la colaboración y su orden (InOrder); el resultado depende de otros.

PU-015: HistorialAcciones.deshacerUltima()	
ID:	PU-015
Clase bajo prueba:	HistorialAcciones
Método:	`deshacerUltima()`
Precondición / datos de entrada:	Última acción hecha hace 10 minutos.
Resultado esperado:	La fábrica reconstruye el comando, se llama deshacer(), la acción queda deshecha y se notifica con los puntos de antes.
Tipo de Test Double:	Mock
Justificación del tipo de Test Double:	Se verifica que el invocador delega en la fábrica y en el comando sin conocer su tipo.

PU-016: HistorialAcciones.deshacerUltima()	
ID:	PU-016
Clase bajo prueba:	HistorialAcciones
Método:	`deshacerUltima()`
Precondición / datos de entrada:	Última acción hecha hace 16 minutos; reloj fijo.
Resultado esperado:	ReglaNegocioException accion.fuera-de-ventana; la fábrica no se usa y la acción no cambia.
Tipo de Test Double:	Fake
Justificación del tipo de Test Double:	Un Clock.fixed es un reloj real pero controlado: la prueba es repetible a cualquier hora.

PU-017: FabricaComandosCazador.paraDeshacer()	
ID:	PU-017
Clase bajo prueba:	FabricaComandosCazador
Método:	`paraDeshacer()`
Precondición / datos de entrada:	Una AccionRegistrada de cada TipoAccion (prueba parametrizada).
Resultado esperado:	Devuelve un comando del mismo tipo y del mismo cazador.
Tipo de Test Double:	Dummy
Justificación del tipo de Test Double:	Los receptores se pasan al constructor pero no se usan al solo crear el comando.

PU-018: NotificadorProgreso.notificar()	
ID:	PU-018
Clase bajo prueba:	NotificadorProgreso
Método:	`notificar()`
Precondición / datos de entrada:	Tres observadores; el primero y el tercero devuelven una novedad.
Resultado esperado:	Los avisa en orden con el mismo evento (cazador, puntos de antes, hora) y concatena las novedades.
Tipo de Test Double:	Mock
Justificación del tipo de Test Double:	Se verifica el orden de las llamadas, que es una regla del diseño.

PU-019: SelectorCriterioOrden.para()	
ID:	PU-019
Clase bajo prueba:	SelectorCriterioOrden
Método:	`para()`
Precondición / datos de entrada:	Selector con Cercanía y Novedad; se pide Novedad y luego Popularidad; otro selector con Cercanía repetida.
Resultado esperado:	Entrega OrdenPorNovedad; falla para Popularidad; rechaza dos estrategias del mismo tipo.
Tipo de Test Double:	Ninguno
Justificación del tipo de Test Double:	Las estrategias reales son baratas de crear.

PU-020: SincronizadorInsignias.alCambiarProgreso()	
ID:	PU-020
Clase bajo prueba:	SincronizadorInsignias
Método:	`alCambiarProgreso()`
Precondición / datos de entrada:	El cazador tenía Retador y su progreso actual solo merece Primer hallazgo.
Resultado esperado:	Borra Retador, guarda Primer hallazgo y reporta las dos novedades.
Tipo de Test Double:	Stub y Mock
Justificación del tipo de Test Double:	Stub para el progreso y las insignias guardadas; Mock para verificar el borrado y el guardado.

PU-021: ServicioRanking.clasificacion()	
ID:	PU-021
Clase bajo prueba:	ServicioRanking
Método:	`clasificacion()`
Precondición / datos de entrada:	Tres cazadores con actividad y uno sin puntos; dos empatan en puntos y cazas.
Resultado esperado:	Orden lula_bites, brasa_norte, humo_azul con puestos 1, 2, 2; el que no tiene puntos no aparece.
Tipo de Test Double:	Stub
Justificación del tipo de Test Double:	Las proyecciones de los repositorios se reemplazan por datos fijos.

PU-022: ServicioCuentas.registrarCazador()	
ID:	PU-022
Clase bajo prueba:	ServicioCuentas
Método:	`registrarCazador()`
Precondición / datos de entrada:	El correo (con mayúsculas y espacios) ya existe.
Resultado esperado:	ReglaNegocioException cuenta.correo-en-uso; la clave nunca se cifra y no se guarda nada.
Tipo de Test Double:	Stub y Dummy
Justificación del tipo de Test Double:	Stub del repositorio que dice que el correo existe; el cifrador es Dummy y se verifica que no se usa.

PU-023: RedireccionSegura.haciaOrigen()	
ID:	PU-023
Clase bajo prueba:	RedireccionSegura
Método:	`haciaOrigen()`
Precondición / datos de entrada:	Referer del mismo sitio, de otro dominio, mal formado y ausente.
Resultado esperado:	Vuelve a la ruta local solo en el primer caso; en los demás usa la ruta por defecto.
Tipo de Test Double:	Fake
Justificación del tipo de Test Double:	MockHttpServletRequest es una implementación funcional y liviana de la petición HTTP.

ID	Clase / Método	Entrada	Resultado esperado	Test Double	CU relacionado
PU-001	CalculadoraPuntos (cadena de decoradores).calcular()	Cocina legendaria, primer cazador, 7 días de racha, 11:00 p. m.	525 puntos con desglose en orden: base 250, primer cazador 50, racha 50 (tope de 5 días), hora fantasma 175.	Ninguno	CU-01
PU-002	BonoPrimerCazador.calcular()	Un BonoPrimerCazador que envuelve un PuntosBasePorRareza espiado; cocina rara, no es primer cazador.	La calculadora envuelta se llama exactamente una vez y el puntaje queda en 150 sin bono.	Spy	CU-01
PU-003	RachaDiaria.diasConsecutivosHasta()	Cazas ayer, dos veces antier y hace 4 días; hoy 10 de noviembre.	2 días: cuenta hasta ayer, ignora repetidos y se corta en el primer hueco.	Ninguno	CU-01
PU-004	CocinaFantasma.estaAbierta()	Horarios 12:00 a 23:00 y 18:00 a 02:00 con horas en los bordes (prueba parametrizada de 8 casos).	Abierta desde la apertura incluida hasta antes del cierre, también cruzando la medianoche.	Ninguno	CU-01, CU-04
PU-005	CocinaFantasma.Builder.build()	Builder sin canales, con apertura igual al cierre, nombre en blanco o sin fecha de alta.	IllegalArgumentException en cada caso; con datos válidos queda activa y con el nombre sin espacios.	Ninguno	CU-09
PU-006	Nivel.paraPuntos()	Puntos en cada límite: 0, 299, 300, 799, 800, 1600, 2999, 3000 y 99999.	Novato, Novato, Explorador, Explorador, Rastreador, Cazador élite, Cazador élite, Leyenda, Leyenda.	Ninguno	CU-06
PU-007	Descubrimiento.resenar()	Un descubrimiento que ya tiene una reseña recibe una segunda.	ReglaNegocioException con código resena.duplicada.	Ninguno	CU-03
PU-008	ParticipacionReto.actualizarAvance()	Reto con meta 4; avance 3, luego 4, luego 5 y después 3.	Solo devuelve true al pasar a completado; conserva la fecha de la primera vez y se revierte al bajar de la meta.	Ninguno	CU-05
PU-009	AccionRegistrada.sePuedeDeshacer()	Acción de las 9:00 p. m. consultada a los 15 min, a los 15 min y 1 s, y después de deshecha.	true, false y false.	Ninguno	CU-02
PU-010	ProgresoCazador y TipoInsignia.desde() y laMerece()	Dos cazas: una legendaria en hora fantasma con reseña y otra común de día; 1 reto completado.	Progreso (2, 2, 2, 1, 1, 1, 1); merece Primer hallazgo, Noctámbulo, Leyenda local y Retador, y no las demás.	Ninguno	CU-06
PU-011	OrdenPorCalificacion.ordenar()	Tres fichas: dos con 4,5 estrellas (6 y 1 reseñas) y una con 4,0.	Primero la de 4,5 con más reseñas, luego la otra de 4,5 y al final la de 4,0.	Ninguno	CU-04
PU-012	RegistroCaza.descubrir()	Cocina que abre de 6:00 a 14:00 consultada a las 9:30 p. m.	ReglaNegocioException caza.cocina-cerrada; la calculadora nunca se usa y no se guarda nada.	Stub y Dummy	CU-01
PU-013	RegistroCaza.descubrir()	El repositorio indica 5 cazas hoy para el cazador.	ReglaNegocioException caza.limite-diario.	Stub	CU-01
PU-014	HistorialAcciones.ejecutar()	Comando que devuelve 150 puntos; el cazador tenía 280 antes.	En orden: ejecuta el comando, guarda la AccionRegistrada con 150 puntos y la hora del reloj, y notifica con 280.	Mock	CU-01, CU-03
PU-015	HistorialAcciones.deshacerUltima()	Última acción hecha hace 10 minutos.	La fábrica reconstruye el comando, se llama deshacer(), la acción queda deshecha y se notifica con los puntos de antes.	Mock	CU-02
PU-016	HistorialAcciones.deshacerUltima()	Última acción hecha hace 16 minutos; reloj fijo.	ReglaNegocioException accion.fuera-de-ventana; la fábrica	Fake	CU-02

ID	Clase / Método	Entrada	Resultado esperado	Test Double	CU relacionado
			no se usa y la acción no cambia.		
PU-017	FabricaComandosCazador.paraDeshacer()	Una AccionRegistrada de cada TipoAccion (prueba parametrizada).	Devuelve un comando del mismo tipo y del mismo cazador.	Dummy	CU-02
PU-018	NotificadorProgreso.notificar()	Tres observadores; el primero y el tercero devuelven una novedad.	Los avisa en orden con el mismo evento (cazador, puntos de antes, hora) y concatena las novedades.	Mock	CU-06
PU-019	SelectorCriterioOrden.para()	Selector con Cercanía y Novedad; se pide Novedad y luego Popularidad; otro selector con Cercanía repetida.	Entrega OrdenPorNovedad; falla para Popularidad; rechaza dos estrategias del mismo tipo.	Ninguno	CU-04
PU-020	SincronizadorInsignias.alCambiarProgreso()	El cazador tenía Retador y su progreso actual solo merece Primer hallazgo.	Borra Retador, guarda Primer hallazgo y reporta las dos novedades.	Stub y Mock	CU-06
PU-021	ServicioRanking.clasificacion()	Tres cazadores con actividad y uno sin puntos; dos empatan en puntos y cazas.	Orden lula_bites, brasa_norte, humo_azul con puestos 1, 2, 2; el que no tiene puntos no aparece.	Stub	CU-08
PU-022	ServicioCuentas.registrarCazador()	El correo (con mayúsculas y espacios) ya existe.	ReglaNegocioException cuenta.correo-en-uso; la clave nunca se cifra y no se guarda nada.	Stub y Dummy	CU-10
PU-023	RedireccionSegura.haciaOrigen()	Referer del mismo sitio, de otro dominio, mal formado y ausente.	Vuelve a la ruta local solo en el primer caso; en los demás usa la ruta por defecto.	Fake	CU-02, CU-10

6.2 Pruebas de Integración

Requisito mínimo: Mínimo 5 escenarios que cubran flujos end-to-end entre componentes.

ID	Flujo / CU cubierto	Componentes involucrados	Resultado esperado	Estrategia de aislamiento
PI-001	Cazar una cocina abierta (CU-01, CU-06)	MockMvc, Spring Security, CocinaControlador, HistorialAcciones, RegistroCaza, cadena de decoradores, observadores, JPA y H2	Redirección a la ficha con la comanda; descubrimiento de 200 puntos; acción en el historial; insignia Primer hallazgo.	@SpringBootTest con perfil test, H2 en memoria, reloj fijo con @Primary y transacción revertida al final.
PI-002	Deshacer la última caza (CU-02, CU-06)	AccionesControlador, HistorialAcciones, FabricaComandos, RegistroCaza, observadores	La caza desaparece, la acción queda deshecha, la insignia se retira y se vuelve a la página de origen.	Igual que PI-001; la caza previa se hace por HTTP en la misma prueba.
PI-003	Regla incumplida al cazar (CU-01)	CocinaControlador, ManejadorErroresWeb, messages.properties	Redirección con el mensaje «La cocina está cerrada…» y sin datos guardados.	Cocina cerrada creada por el Escenario de prueba; reloj fijo a las 9:30 p. m.
PI-004	Completar un reto y verlo en el ranking (CU-05, CU-08)	RetosControlador, UnirseARetoComando, SincronizadorRetos, ServicioRanking, API REST	El reto queda completado, los puntos suman 520 y la API del ranking lo muestra primero con nivel Explorador.	Datos propios de la prueba; rollback al final.
PI-005	Acceso por rol y CSRF (CU-10)	Cadena de filtros de Spring Security, RedireccionPorRol, ServicioDetallesUsuario	Anónimo va a /ingresar; cazador recibe 403 en /admin; POST sin token CSRF recibe 403; el ingreso redirige por rol.	Usuarios simulados con user() y un formLogin real contra la base H2.
PI-006	API REST del catálogo (CU-04, CU-08)	CocinasApi, ServicioCatalogo, ManejadorErroresApi, Jackson	Filtros y orden iguales al explorador; 404 en formato Problem Details para cocinas inactivas; 400 con id inválido.	MockMvc sin sesión; cocinas activas, cerradas e inactivas creadas para la prueba.
PI-007	Registro de cazador (CU-10)	CuentaControlador, Bean Validation, ServicioCuentas,	Redirección a /ingresar?registrado; clave guardada con	Rollback por prueba; sin datos de demostración

ID	Flujo / CU cubierto	Componentes involucrados	Resultado esperado	Estrategia de aislamiento
		BCrypt, RepositorioUsuarios	BCrypt; correo repetido marcado en el campo; errores por campo.	(perfil test).
PI-008	Gestión del catálogo y de retos (CU-09)	AdminCocinasControlador, AdminRetosControlador, AdministracionCocinas, Builder, ServicioRetos	Cocina válida registrada; errores de horario, coordenadas y canales; no se elimina una cocina con cazas; un reto sin inscritos sí.	Administrador simulado; datos propios.
PI-009	Renderizado de todas las vistas (CU-04, CU-07 y demás)	Thymeleaf, todas las plantillas, AtributosComunes, TraductorNovedades	Cada página responde 200 (o 404 en la cocina inexistente) y contiene los datos esperados, incluida la barra de deshacer con 900 segundos.	Escenario completo creado por HTTP en @BeforeEach.
PI-010	Consultas de los repositorios (CU-06, CU-08)	Spring Data JPA, JPQL con proyecciones, Hibernate y H2	Sumas de puntos, conteo de reseñas, promedio de calificaciones y restricción única cazador y cocina.	@DataJpaTest: solo la capa de persistencia con una base embebida nueva.
PI-011	Datos de demostración	CargadorDatosDemo, cadena de decoradores, AvanceRetos y observadores reales	16 cocinas, 6 cazadores y 36 cazas que respetan horarios y el límite diario; racha y retos esperados; la carga no se duplica.	Contexto aparte con perfil demo y reloj del sistema.

Además, ArquitecturaTest (ArchUnit) verifica en cada build siete reglas: sentido de las capas, dominio sin Spring, presentación sin entidades, ausencia de ciclos, comandos encapsulados en su paquete, observadores con @Order e inyección solo por constructor.

6.3 Cobertura Objetivo

Porcentaje de cobertura comprometido para clases del dominio	Justificación de la cifra elegida
≥ 90 % de líneas en dominio.* (≥ 70 % de líneas y de ramas en todo el proyecto)	El dominio concentra las reglas del juego (puntos, horarios, retos, insignias, niveles y la ventana de deshacer) y se prueba sin Spring, así que es barato cubrirlo casi por completo. No se exige 100 % porque los constructores protegidos que pide JPA no tienen lógica. El umbral está en el pom: el plugin de JaCoCo falla el build si no se cumple, sumando pruebas unitarias y de integración.

Sección 7 — Métricas de Calidad y Plan de Refactorización

Las métricas de calidad deben definirse ANTES de implementar, no al final. Establecer umbrales numéricos concretos compromete al equipo con un estándar medible durante el desarrollo.

7.1 Métricas Seleccionadas

Requisito mínimo: Mínimo 4 métricas. Al menos 3 deben ser medibles con SonarQube.

Métrica	Herramienta	Umbral aceptable	Justificación del umbral
Cobertura de código	SonarQube / JaCoCo	≥ 70%	Valor del curso para todo el proyecto, con 90 % para el dominio. Lo verifican el check de JaCoCo y la puerta de calidad de SonarQube.
Complejidad ciclomática	SonarQube	≤ 10 por método	Más de 10 caminos hace un método difícil de probar. PMD falla el build desde 11 y el perfil de calidad propio de SonarQube activa java:S1541 con máximo 10.

Métrica	Herramienta	Umbral aceptable	Justificación del umbral
Deuda técnica	SonarQube	≤ 30 min / KLOC	SonarQube no tiene esta métrica: con su costo por defecto de 30 min por línea equivale a un sqale_debt_ratio de 0,1 %, que es la condición de la puerta de calidad. El flujo de CI guarda además el valor en minutos por KLOC.
Duplicación de código	SonarQube	≤ 3%	Con Decorator, Strategy y la clase de parámetros compartida no debería repetirse lógica. CPD la revisa en cada build.
Complejidad cognitiva (adicional)	SonarQube / PMD	≤ 15 por método	Mide lo difícil que es leer un método (anidamiento y saltos). 15 es el valor por defecto de la regla de SonarQube.
Bugs y vulnerabilidades (adicional)	SonarQube	0 (calificación A)	Un juego con cuentas y puntos no debe tener fallas de fiabilidad ni de seguridad abiertas.
Violaciones de arquitectura (adicional)	ArchUnit	0	Las capas solo protegen el diseño si se verifican en cada build.

7.2 Plan de Refactorización Anticipado

Identificar code smells que el equipo anticipa en el diseño actual y la técnica de refactorización que se aplicará para eliminarlos.

Requisito mínimo: Mínimo 5 code smells con técnica de refactorización y efecto esperado.

Code smell anticipado	Dónde se anticipa	Técnica de refactorización	Efecto esperado en métricas
Long Method	RegistroCaza.descubrir(): validaciones, contexto, cálculo y guardado juntos.	Extract Method (validar() y diasDeRacha()).	Complejidad ciclomática por método por debajo de 10.
Switch Statements	Cálculo de bonos con if/else por concepto y orden del explorador por tipo.	Replace Conditional with Polymorphism (Decorator y Strategy).	Menor complejidad cognitiva; agregar un bono no aumenta la de nadie.
Long Parameter List	Constructor de CocinaFantasma con diez parámetros.	Replace Constructor with Builder e Introduce Parameter Object (DatosCocina).	Menos code smells en SonarQube y validación en un solo punto.
Data Clumps	Latitud, longitud y barrio viajando juntos por varias clases.	Introduce Parameter Object (Ubicacion y Coordenada).	Menos duplicación y fórmula de distancia en un solo lugar.
Duplicated Code	Lectura de filtros en ExplorarControlador y en CocinasApi.	Extract Class (ParametrosBusqueda).	Duplicación por debajo del 3 % y CPD sin hallazgos.
Feature Envy	ServicioRanking calculando puntos con datos de reseñas y retos.	Move Method al dominio (PuntosCazador.total()).	Una sola fórmula para panel y ranking; menos deuda técnica.
Large Class	CocinaControlador si crece con más acciones sobre la cocina.	Extract Class (por ejemplo un controlador solo para reseñas).	Menos acoplamiento y regla GodClass en verde.